PYTEST

CHEAT SHEETS

Testing Python Code with pytest 9

Configuration · Discovery · assert · pytest.raises · Fixtures · Scopes · conftest.py · Built-in Fixtures · Parametrize · Subtests · Marks · monkeypatch · pytest-mock · CLI · Debugging · Plugins & Hooks · Coverage · xdist · Async · Django · Hypothesis · CI

25 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & First Test
02 Configuration
03 Discovery & Project Layout

ASSERTIONS

04 Assertions & Failure Output
05 Exceptions & Warnings

FIXTURES

06 Fixture Basics
07 Fixture Scope & Teardown
08 conftest.py & Sharing Fixtures
09 Built-in Fixtures

PARAMETRIZE, SUBTESTS & MARKS

10 Parametrize
11 Subtests
12 Marks: skip, xfail & Custom

MOCKING

13 monkeypatch
14 unittest.mock & pytest-mock

RUNNING & DEBUGGING

15 Selecting & Rerunning Tests
16 Output & Debugging

PLUGINS IN PRACTICE

17 Plugins & Hooks
18 Coverage with pytest-cov
19 Parallel & Flaky Tests
20 Async Tests
21 Django & Web Apps

PATTERNS & CI

22 Testing Patterns in pytest
23 pytest in CI

QUICK REFERENCE

24 Quick Reference — Fixtures, Marks & Assertions
25 Quick Reference — CLI & Config

PYT

Table of Contents

25 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 03

- 01 Introduction & First Test
- 02 Configuration
- 03 Discovery & Project La…

ASSERTIONS 04 – 05

- 04 Assertions & Failure O…
- 05 Exceptions & Warnings

FIXTURES 06 – 09

- 06 Fixture Basics
- 07 Fixture Scope & Teard…
- 08 conftest.py & Sharing …
- 09 Built-in Fixtures

PARAMETRIZE, SUBTESTS & MARKS 10 – 12

- 10 Parametrize
- 11 Subtests
- 12 Marks: skip, xfail & Cus…

MOCKING 13 – 14

- 13 monkeypatch
- 14 unittest.mock & pytest…

RUNNING & DEBUGGING 15 – 16

- 15 Selecting & Rerunning …
- 16 Output & Debugging

PLUGINS IN PRACTICE 17 – 21

- 17 Plugins & Hooks
- 18 Coverage with pytest-…
- 19 Parallel & Flaky Tests
- 20 Async Tests
- 21 Django & Web Apps

PATTERNS & CI 22 – 23

- 22 Testing Patterns in pyt…
- 23 pytest in CI

QUICK REFERENCE 24 – 25

- 24 Quick Reference — Fix…
- 25 Quick Reference — CLI…

Sheet 01

Introduction & First Test

why pytest · `pip install pytest` · `test_*.py` · plain `assert` · `pytest` command · reading the output

PLAIN FUNCTIONS, PLAIN ASSERT

pytest is the most widely used Python test runner. A test is a function whose name starts with `test_`, in a file named `test_*.py`, that uses the ordinary `assert` statement. pytest finds the tests, runs them, and when an assert fails it shows the values on both sides. No classes or special assert methods are needed, though pytest also runs existing `unittest` tests unchanged.

NOTES

- Install into the project's environment: `pip install pytest` or `uv add --dev pytest`
- Run from the project root: `pytest`
- Each `.` is a pass, `F` a failure, `E` an error in setup, `s` a skip
- Exit code 0 = all passed; anything else fails CI

A FIRST TEST FILE

```python
# src/shop/cart.py
def total(prices, discount=0.0):
    if not 0 <= discount < 1:
        raise ValueError("discount must be 0..1")
    return round(sum(prices) * (1 - discount), 2)

# tests/test_cart.py
from shop.cart import total

def test_total_adds_prices():
    assert total([2.50, 1.25]) == 3.75

def test_total_applies_discount():
    assert total([10, 10], discount=0.25) == 15.0

def test_empty_cart_is_zero():
    assert total([]) == 0

# $ pytest
# tests/test_cart.py ...                    [100%]
# ============ 3 passed in 0.02s ============
```

CODE EXPLAINED

→ Test files start with `test_` (or end in `_test.py`); test functions start with `test_`

→ Each test checks one behavior and reads like a sentence

→ pytest imports the test module, so the code under test must be importable (sheet 03)

→ The progress line shows one character per test, then a summary

UNITTEST EQUIVALENT

≈ `def test_x():` + `assert` → a `unittest.TestCase` subclass with `def test_x(self):` and `self.assertEqual`

≈ `pytest` → `python -m unittest discover`

≈ pytest runs `unittest` classes as they are, so you can migrate gradually

PREFER / AVOID

⇄ `if result != 3.75: raise AssertionError` → `assert result == 3.75`

⇄ one test that checks five things → five small tests with clear names

⇄ `print` to check results by eye → an `assert` that fails on its own

OUTPUT CHARACTERS

CHAR	MEANS
`.`	Passed
`F`	Failed assertion
`E`	Error in setup / teardown or collection
`s`	Skipped
`x`	Expected failure (xfail)
`X`	Unexpectedly passed (xpass)

EXIT CODES

CODE	MEANING
`0`	All tests passed
`1`	Some tests failed
`2`	Interrupted (Ctrl+C, errors in collection)
`3`	Internal error
`4`	Usage error (bad option)
`5`	No tests collected

⚠ COMMON MISTAKES

- Naming a file `cart_test_utils.py` or a function `check_total`; pytest won't collect them
- Installing pytest globally and running it against the wrong Python environment
- Putting several unrelated checks in one test, so the first failure hides the rest

✓ TIPS & BEST PRACTICES

- Run `python -m pytest` if `pytest` picks up the wrong interpreter; it also adds the current folder to `sys.path`
- Keep tests in a `tests/` folder that mirrors the package layout
- Name tests after the behavior: `test_empty_cart_is_zero`

QUICK SUMMARY

- `test_*.py` + `def test_*` — collected automatically
- `assert` — plain Python, rich failure output
- `pytest` — run from the project root
- exit code — 0 means everything passed

Configuration

ONE CONFIG BLOCK, READ ON EVERY RUN

pytest reads its settings from the first config file it finds. **pytest 9** reads native TOML: a `[tool.pytest]` table in `pyproject.toml`, or a `pytest.toml` / `.pytest.toml` file. Older projects use `[tool.pytest.ini_options]` or `pytest.ini`, which still work. Put options you always want (`testpaths`, `addopts`, markers) here instead of typing them each run.

NOTES

- The directory holding the config file becomes the **rootdir**
- `pytest --help` lists every config key under "[pytest] ini-options"
- `addopts` flags are added before the ones you type
- `strict = true` turns on all the strict checks at once (9.0)

UNDER THE HOOD

- pytest walks up from the given paths looking for `pytest.toml`, `.pytest.toml`, `pytest.ini`, `pyproject.toml` (with a pytest table), `tox.ini`, `setup.cfg`
- The first match wins; files are not merged
- `pytest -c path/to/file` forces a config file; the header line `configfile:` shows which one was used

PYPROJECT.TOML (PYTEST 9 NATIVE TOML)

```toml
[tool.pytest]
minversion = "9.0"
testpaths = ["tests"]
addopts = ["-ra", "--strict-markers", "--import-mode=importlib"]
strict = true
markers = [
  "slow: takes more than a second",
  "integration: needs the database",
]
filterwarnings = ["error", "ignore::DeprecationWarning:somelib.*"]
log_cli_level = "INFO"

# ── pytest 8 and older ──────────────────────
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-ra --strict-markers"
```

CODE EXPLAINED

→ In native TOML, list options are real arrays; `ini_options` takes strings like an INI file

→ `testpaths` sets where to look when you run plain `pytest`

→ `filterwarnings = ["error"]` turns warnings into failures, with exceptions listed after it

→ `minversion` fails fast if someone runs an older pytest

UNITTEST EQUIVALENT

≈ config file → none built in; you pass options to `python -m unittest` each time

≈ `testpaths` → `python -m unittest discover -s tests`

≈ `filterwarnings` → `python -W error -m unittest`

PREFER / AVOID

⇄ typing `-ra --strict-markers` every run → put them in `addopts`

⇄ `setup.cfg` / `tox.ini` sections → `[tool.pytest]` in `pyproject.toml`

⇄ unregistered markers → register them under `markers` and use `--strict-markers`

CONFIG FILES (PRIORITY ORDER)

FILE	SECTION
`pytest.toml / .pytest.toml`	top level (9.0+)
`pytest.ini / .pytest.ini`	[pytest]
`pyproject.toml`	[tool.pytest] (9.0+) or [tool.pytest.ini_options]
`tox.ini`	[pytest]
`setup.cfg`	[tool:pytest]

COMMON KEYS

KEY	SETS
`testpaths`	Folders to search
`addopts`	Default CLI flags
`markers`	Registered marks
`python_files`	Test file patterns
`norecursedirs`	Folders to skip
`filterwarnings`	Warning filters
`log_cli`	Live logging
`strict`	All strict checks (9.0)

⚠ COMMON MISTAKES

- Writing `[tool.pytest]` with pytest 8, which silently ignores it
- Having both `pytest.ini` and `pyproject.toml` settings and editing the one that isn't used
- Putting environment-specific flags (like `-x`) in `addopts`, so CI stops at the first failure

✓ TIPS & BEST PRACTICES

- Check the `configfile:` line at the top of the output to see which file was read
- Use `strict = true` on new projects so typos in markers and config keys fail
- Keep `addopts` to flags everyone wants; personal ones go on the command line or in `PYTEST_ADDOPTS`

QUICK SUMMARY

- `[tool.pytest]` — native TOML (9.0+)
- `ini_options` — older pytest
- `testpaths` / `addopts` — defaults for every run
- first config file found wins

Discovery & Project Layout

HOW PYTEST FINDS AND IMPORTS TESTS

pytest **collects** tests by walking the folders you give it (or `testpaths`), importing files that match `test_*.py` or `*_test.py`, then gathering functions named `test*` and methods named `test*` in classes named `Test*` (with no `__init__`). The common modern layout puts code in `src/package/` and tests in `tests/`, installed with `pip install -e .`, and uses `--import-mode=importlib` so test files don't need `__init__.py`.

NOTES

- `pytest --co -q` lists what would run without running it
- Test classes group tests; they must not define `__init__`
- Collection errors (`E` before any test runs) are usually import errors
- Change patterns with `python_files`, `python_classes`, `python_functions`

UNDER THE HOOD

- Collection walks `testpaths` → directories → modules → classes → functions, building **items**
- With the default `prepend` mode, each test file's folder goes on `sys.path`, so two `test_utils.py` files in different folders clash
- `importlib` mode imports by path without touching `sys.path` and avoids the clash

SRC LAYOUT

```
orders/
├── pyproject.toml          # [tool.pytest] testpaths = ["tests"]
├── src/
│   └── orders/
│       ├── __init__.py
│       ├── cart.py
│       └── payments.py
└── tests/
    ├── conftest.py         # shared fixtures (sheet 08)
    ├── test_cart.py
    └── payments/
        └── test_refunds.py

$ pip install -e .          # makes "import orders" work
$ pytest --co -q            # list collected tests
tests/test_cart.py::test_total_adds_prices
tests/test_cart.py::TestDiscount::test_quarter_off
tests/payments/test_refunds.py::test_full_refund
```

CODE EXPLAINED

→ The **src layout** stops tests from accidentally importing the local folder instead of the installed package

→ The node ID `file::Class::test` is what you pass to run a single test (sheet 15)

→ Nested test folders don't need `__init__.py` with `--import-mode=importlib`

→ `conftest.py` files are picked up automatically wherever they sit

UNITTEST EQUIVALENT

≈ `test_*.py` discovery → `unittest discover -p "test*.py"`

≈ test functions → must be methods on a `TestCase` subclass

≈ `--co` → no equivalent; `unittest` has no collect-only mode

PREFER / AVOID

⇄ `sys.path.insert(0, "..")` at the top of test files → src layout + `pip install -e .`

⇄ duplicate `test_utils.py` with prepend mode → `--import-mode=importlib`

⇄ a `TestThing` class with `__init__` → plain functions or a fixture

WHAT GETS COLLECTED

ITEM	DEFAULT PATTERN
Files	test_*.py, *_test.py
Classes	Test* (no __init__)
Functions	test* (module level)
Methods	test* inside Test*
unittest	Any TestCase subclass

IMPORT MODES

MODE	BEHAVIOR
prepend (default)	Adds the test file's rootdir to the front of sys.path
append	Adds it to the end
importlib	Imports by path, no sys.path changes; recommended for new projects

⚠ COMMON MISTAKES

- Defining `__init__` on a test class; pytest skips the whole class with a warning
- Running tests against the source folder instead of the installed package and missing packaging bugs
- Ignoring collection errors and only reading the "passed" count

✓ TIPS & BEST PRACTICES

- Run `pytest --co -q` after moving files to confirm everything is still collected
- Use `pythonpath = ["src"]` in config if you can't install the package in editable mode
- Mirror the package's folders under `tests/` so tests are easy to find

QUICK SUMMARY

- `test_*.py` / `Test*` / `test_*` — collected
- src layout + `pip install -e .`
- `--import-mode=importlib` — no `__init__.py` needed
- `--co` — list without running

Assertions & Failure Output

PLAIN ASSERT, DETAILED FAILURES

pytest **rewrites** `assert` statements in test modules so a failure prints every sub-expression's value: both sides of ==, the differing dict keys, the first differing list index. Add a message after a comma for context. Floating-point results should use **`pytest.approx`**, which compares within a tolerance instead of exactly.

NOTES

- `-vv` prints full diffs instead of truncated ones
- Compare whole structures (`== {...}`) to get a key-by-key diff
- `approx` works on numbers, lists, dicts and NumPy arrays
- Helper modules need `pytest.register_assert_rewrite` to get the same output

UNDER THE HOOD

- Assertion rewriting happens at **import time**: pytest's import hook rewrites the AST of test modules and `conftest.py`
- Plain helper modules imported by tests are not rewritten, so their asserts show only `AssertionError`
- Call `pytest.register_assert_rewrite("tests.helpers")` in `conftest.py` before importing them

ASSERTS AND THEIR FAILURE OUTPUT

```python
def test_user_dict():
    assert get_user(1) == {"id": 1, "name": "Ann", "admin": False}
# E  AssertionError: assert {...} == {...}
# E    Differing items:
# E    {'admin': True} != {'admin': False}

def test_list():
    assert sorted(tags()) == ["a", "b", "c"]
# E    At index 1 diff: 'z' != 'b'

def test_with_message():
    items = cart.items()
    assert len(items) == 2, f"cart had {items!r}"

def test_float():
    assert 0.1 + 0.2 == pytest.approx(0.3)
    assert total() == pytest.approx(19.99, abs=0.01)
    assert [0.1 + 0.2, 0.3] == pytest.approx([0.3, 0.3])

def test_membership_and_identity():
    assert "error" not in log_text
    assert result is None
```

CODE EXPLAINED

- → Comparing whole dicts shows only the differing keys, which is clearer than asserting each key
- → The message after the comma prints under the failure; use it for context the values don't show
- → `approx` defaults to a relative tolerance of 1e-6; set `abs=` or `rel=` explicitly for money and sensors
- → `is None` checks identity, which is right for `None`, `True` and `False`

UNITTEST EQUIVALENT

- ≈ `assert a == b` → `self.assertEqual(a, b)`
- ≈ `assert x in y` → `self.assertIn(x, y)`
- ≈ `pytest.approx` → `self.assertAlmostEqual(a, b, places=7)`
- ≈ `assert x is None` → `self.assertIsNone(x)`

PREFER / AVOID

- ⇄ `assert a == b == c` chains that hide which one failed → separate asserts
- ⇄ `assert 0.1 + 0.2 == 0.3` → `== pytest.approx(0.3)`
- ⇄ `assert len(x) > 0` → `assert x` when emptiness is the point

COMMON ASSERT FORMS

WRITE	CHECKS
`assert a == b`	Equality (rich diff)
`assert x`	Truthy
`assert not x`	Falsy
`assert x in y`	Membership
`assert isinstance(x, T)`	Type
`assert x is None`	Identity
`== pytest.approx(v)`	Float within tolerance

APPROX OPTIONS

OPTION	EXAMPLE
`rel`	approx(100, rel=0.01) → 99..101
`abs`	approx(0, abs=1e-9)
`Lists / dicts`	approx([0.3, 0.6])
`nan_ok`	approx(nan, nan_ok=True)

⚠ COMMON MISTAKES

- Wrapping a comparison in parentheses with a message: `assert (a == b, "msg")` is always true (a tuple)
- Comparing floats with `==` and getting random failures
- Asserting inside a helper module and getting a bare `AssertionError` with no values

✓ TIPS & BEST PRACTICES

- Assert on whole objects or dicts to get one readable diff
- Use `-vv` when a diff is truncated with `...Full output truncated`
- Use `assert result == expected` with `expected` named in the test, so the failure reads well

QUICK SUMMARY

- `assert` — rewritten for rich output
- message — `assert cond, "context"`
- `pytest.approx` — floats within tolerance
- `-vv` — full diffs

Exceptions & Warnings

TEST THAT CODE FAILS CORRECTLY

pytest.raises is a context manager that passes only if the block raises the given exception type (or a subclass). `match=` checks the message with a regex, and the returned **ExceptionInfo** gives access to the exception object for further asserts. **pytest.warns** does the same for warnings, and **pytest.RaisesGroup** checks an `ExceptionGroup` from `asyncio.TaskGroup` or `except*` code.

NOTES

- Keep only the line that should raise inside the `with` block
- `match` uses `re.search`, so escape special characters with `re.escape`
- Pass a tuple to accept several types: `raises((KeyError, IndexError))`
- `filterwarnings = ["error"]` in config makes unexpected warnings fail

RAISES, WARNS AND GROUPS

```python
import re, warnings, pytest

def test_bad_discount():
    with pytest.raises(ValueError, match=r"discount must be 0\.\.1"):
        total([10], discount=2)

def test_inspect_exception():
    with pytest.raises(HTTPError) as excinfo:
        client.get("/missing")
    assert excinfo.value.status == 404
    assert excinfo.type is NotFound

def test_literal_message():
    with pytest.raises(KeyError, match=re.escape("['sku']")):
        load({"name": "x"})

def test_task_group():
    with pytest.RaisesGroup(ValueError, TypeError):
        run_all_jobs()

def test_deprecated():
    with pytest.warns(DeprecationWarning, match="use total"):
        old_total([1])
    with pytest.deprecated_call():
        old_total([1])
```

CODE EXPLAINED

→ `excinfo.value` is the exception instance; `excinfo.type` its class

→ Escape regex characters in `match`, or wrap the text in `re.escape`

→ `RaisesGroup(ValueError, TypeError)` passes when the group holds exactly those exceptions

→ `deprecated_call` accepts `DeprecationWarning` or `PendingDeprecationWarning`

UNITTEST EQUIVALENT

≈ `with pytest.raises(E):` → `with self.assertRaises(E):`

≈ `match=` → `self.assertRaisesRegex(E, pattern)`

≈ `pytest.warns(W)` → `self.assertWarns(W)`

≈ `excinfo.value` → `cm.exception`

PREFER / AVOID

⇄ `try: … except E: pass` + `assert False` → `with pytest.raises(E):`

⇄ a whole test body inside `raises` → only the call that should raise

⇄ `raises(Exception)` → the specific exception type

API

CALL	PASSES WHEN
`raises(E)`	Block raises E or a subclass
`raises(E, match=r"…")`	…and the message matches
`RaisesGroup(E1, E2)`	ExceptionGroup with exactly these
`RaisesExc(E, match=…)`	One expected member inside a group
`warns(W)`	Block emits warning W
`deprecated_call()`	Any deprecation warning

EXCEPTIONINFO

ATTRIBUTE	GIVES
`.value`	Exception instance
`.type`	Exception class
`.match(r"…")`	Assert message after the block
`.traceback`	Traceback entries
`.group_contains(E)`	Check a group member

⚠ COMMON MISTAKES

- Catching `Exception` in `raises`, so a typo-caused `NameError` makes the test pass
- Putting several statements in the block, so the wrong one raises and the test still passes
- Forgetting that `match` is a regex: `match="cost (USD)"` never matches the parentheses literally

✓ TIPS & BEST PRACTICES

- Assert on attributes of `excinfo.value` (status codes, fields) rather than only the message
- Turn warnings into errors in config, then use `warns` where a warning is expected
- Use `RaisesGroup(..., allow_unwrapped=True)` when code may raise a bare exception or a group

QUICK SUMMARY

- `pytest.raises(E, match=)` — expected exception
- `excinfo.value` — inspect it
- `RaisesGroup` — ExceptionGroups
- `warns` / `deprecated_call` — expected warnings

Fixture Basics

SETUP THAT TESTS ASK FOR

A **fixture** is a function decorated with `@pytest.fixture` that builds something a test needs: an object, a temp file, a database connection. A test **requests** it by naming it as a parameter; pytest calls the fixture and passes in the result. Fixtures can request other fixtures, so setup is built from small, reusable pieces instead of one big `setUp` method.

NOTES

- Each test gets a fresh fixture value by default (function scope)
- A fixture runs only for tests that request it (unless `autouse`)
- `pytest --fixtures` lists every available fixture with its docstring
- `@pytest.mark.usefixtures("name")` runs one without passing its value

UNDER THE HOOD

- pytest matches each test parameter **by name** to a registered fixture
- It builds a dependency graph per test and calls fixtures in order, each once per scope
- The `request` fixture gives a fixture access to the test that asked for it (`request.node`, `request.param`)

FIXTURES BUILDING ON FIXTURES

```python
import pytest
from shop.cart import Cart
from shop.catalog import Catalog

@pytest.fixture
def catalog():
    """A catalog with two products."""
    return Catalog({"apple": 0.50, "pear": 0.75})

@pytest.fixture
def cart(catalog):                  # fixture requests a fixture
    return Cart(catalog)

def test_add_item(cart):
    cart.add("apple", qty=4)
    assert cart.total() == 2.00

def test_unknown_item(cart):
    with pytest.raises(KeyError):
        cart.add("kiwi")

@pytest.fixture(autouse=True)
def fixed_clock(monkeypatch):       # runs for every test in scope
    monkeypatch.setattr("shop.cart.now", lambda: 1_700_000_000)

@pytest.mark.usefixtures("clean_db")
def test_checkout(cart): ...
```

CODE EXPLAINED

→ `cart` requests `catalog`, so pytest builds the catalog first and passes it in

→ Each test gets its own `cart`, so tests can't affect each other

→ `autouse=True` applies the fixture to every test in its module (or below its `conftest.py`)

→ The docstring shows up in `pytest --fixtures`

UNITTEST EQUIVALENT

≈ fixture → `setUp(self)` storing `self.cart`

≈ fixture requesting fixture → `setUp` calling helper methods in order

≈ `autouse` → `setUp` in a base class every test class inherits

PREFER / AVOID

⇄ a `setup_method` that builds everything → small fixtures each test requests

⇄ global objects created at module import → fixtures

⇄ calling a fixture function directly → request it as a parameter

@PYTEST.FIXTURE OPTIONS

OPTION	EFFECT
`scope="module"`	Share one value (sheet 07)
`autouse=True`	Apply without requesting
`params=[…]`	Run tests once per value (sheet 08)
`name="db"`	Register under another name
`ids=[…]`	Labels for params

WAYS TO USE A FIXTURE

WAY	GETS THE VALUE?
`Test parameter`	Yes
`Another fixture's parameter`	Yes
`@pytest.mark.usefixtures`	No (setup only)
`autouse=True`	No (setup only)
`request.getfixturevalue("x")`	Yes, dynamically

⚠ COMMON MISTAKES

- Calling `cart()` directly inside a test; pytest raises an error, request it instead
- Making a heavy fixture `autouse` and slowing every test
- Mutating a shared fixture object in one test and relying on it in another

✓ TIPS & BEST PRACTICES

- Keep fixtures small and compose them; it's easier to see what each test needs
- Give fixtures docstrings so `pytest --fixtures` documents them
- Name fixtures as nouns (`cart`, `admin_user`), not verbs

QUICK SUMMARY

- `@pytest.fixture` — setup as a function
- request by parameter name
- fixtures can use fixtures
- `autouse` / `usefixtures` — setup without the value

Fixture Scope & Teardown

HOW LONG A FIXTURE LIVES

Scope decides how often a fixture is built: once per test (`function`, the default), per class, module, package, or once for the whole run (`session`). Expensive resources like a database server or a browser use wider scopes. Put cleanup after a **`yield`**: code before it is setup, the yielded value goes to the test, and code after it runs as teardown even if the test failed.

NOTES

- Teardown runs in reverse order of setup
- A fixture may only request fixtures of the **same or wider** scope
- Shared mutable objects in wide scopes leak state between tests
- `--setup-show` prints every SETUP / TEARDOWN as it happens

UNDER THE HOOD

- pytest caches each fixture value per scope key (test, class, module…) and reuses it until that scope ends
- Teardown is LIFO: the last fixture set up is the first torn down
- A function-scoped fixture requested by a session fixture raises **ScopeMismatch**

YIELD TEARDOWN AND SCOPES

```python
import pytest, sqlite3

@pytest.fixture(scope="session")
def db_path(tmp_path_factory):
    path = tmp_path_factory.mktemp("db") / "test.sqlite"
    create_schema(path)
    return path

@pytest.fixture
def db(db_path):
    conn = sqlite3.connect(db_path)
    conn.execute("BEGIN")
    yield conn                      # ← test runs here
    conn.rollback()                 # teardown, runs even on failure
    conn.close()

@pytest.fixture(scope="module")
def api_server():
    server = start_server(port=0)
    yield server
    server.stop()

def test_insert(db):
    db.execute("INSERT INTO orders VALUES (1, 'ann')")
    assert db.execute("SELECT count(*) FROM orders").fetchone()[0] == 1

# $ pytest --setup-show tests/test_db.py
```

CODE EXPLAINED

→ The schema is built once per run; each test gets a fresh connection inside a transaction

→ Rolling back after each test keeps tests independent while sharing the expensive setup

→ `tmp_path_factory` is the session-scoped version of `tmp_path`

→ `--setup-show` makes scope bugs visible

UNITTEST EQUIVALENT

≈ `function` scope → `setUp` / `tearDown`

≈ `class` scope → `setUpClass` / `tearDownClass` (classmethods)

≈ `module` scope → `setUpModule` / `tearDownModule`

≈ `yield` teardown → `self.addCleanup(fn)`

PREFER / AVOID

⇄ cleanup in a `try/finally` inside each test → `yield` fixture

⇄ session-scoped mutable list shared by tests → function scope or a fresh copy

⇄ `request.addfinalizer` for simple cases → `yield`

SCOPES

SCOPE	BUILT ONCE PER	USE FOR
`function`	Test (default)	Most fixtures
`class`	Test class	Shared state in a class
`module`	Test file	A server per file
`package`	Test package	Per-folder resources
`session`	Whole run	Databases, browsers, containers

TEARDOWN OPTIONS

WAY	WHEN TO USE
`yield`	Default choice
`request.addfinalizer(fn)`	Several cleanups registered conditionally
`Context manager inside yield`	with open(…) as f: yield f

⚠ COMMON MISTAKES

- Requesting a function-scoped fixture (like `monkeypatch`) from a module-scoped one
- Putting teardown before `yield`, or returning instead of yielding
- Session-scoped database without per-test rollback, so test order changes results

✓ TIPS & BEST PRACTICES

- Combine a wide-scope resource with a narrow-scope reset (transaction, truncate)
- Use `scope="session"` for anything that takes seconds to start
- Run `--setup-show` when you suspect a fixture isn't being rebuilt

QUICK SUMMARY

- `scope=` — function, class, module, package, session
- `yield` — setup above, teardown below
- teardown runs LIFO, even on failure
- request only same-or-wider scopes

conftest.py & Sharing Fixtures

FIXTURES EVERY TEST IN A FOLDER CAN USE

Fixtures in a **conftest.py** file are available to every test in that folder and below, with no import. A deeper `conftest.py` can **override** a fixture by defining one with the same name. A **factory fixture** returns a function so a test can build several objects with different arguments. A **parametrized fixture** (`params=`) runs every test that uses it once per value.

NOTES

- Never import from `conftest.py`; pytest loads it for you
- Root `tests/conftest.py` holds project-wide fixtures
- Hooks (sheet 17) also live in `conftest.py`
- Factories avoid writing one fixture per variation

UNDER THE HOOD

- pytest loads `conftest.py` files from the rootdir down to each test's folder before collecting it
- When names clash, the fixture **closest** to the test wins
- Each `conftest.py` is registered as a plugin, which is why it can hold hooks

TESTS/CONFTEST.PY

```python
import pytest
from shop.models import User

@pytest.fixture
def make_user(db):
    """Factory: build users with any fields."""
    created = []
    def _make(name="ann", admin=False, **extra):
        user = User.create(db, name=name, admin=admin, **extra)
        created.append(user)
        return user
    yield _make
    for user in created:            # clean up everything it made
        user.delete()

@pytest.fixture(params=["sqlite", "postgres"], ids=["lite", "pg"])
def db(request):
    conn = connect(request.param)   # runs every test twice
    yield conn
    conn.close()

# tests/admin/conftest.py — override for this folder only
@pytest.fixture
def make_user(make_user):           # wraps the parent fixture
    return lambda **kw: make_user(admin=True, **kw)

def test_two_users(make_user):
    a, b = make_user("ann"), make_user("bo")
    assert a.id != b.id
```

CODE EXPLAINED

→ The factory tracks what it created so teardown can remove it

→ `params=` makes each test using `db` run as `test_x[lite]` and `test_x[pg]`

→ An override can request the fixture it overrides to extend it

→ Tests in `tests/admin/` get the admin-by-default factory

UNITTEST EQUIVALENT

≈ `conftest.py` → a shared `BaseTestCase` class every test inherits

≈ factory fixture → helper methods like `self.make_user()`

≈ `params=` fixture → no direct equivalent; loop with `subTest` or subclass per variant

PREFER / AVOID

⇄ `from conftest import x` → request `x` as a fixture

⇄ one fixture per variant (`admin_user`, `guest_user` …) → a factory fixture

⇄ huge root `conftest.py` → split per folder near the tests that use them

WHERE FIXTURES COME FROM

SOURCE	VISIBLE TO
`Test module`	That module
`conftest.py in a folder`	That folder and below
`Root conftest.py`	All tests
`Plugins (pytest-django…)`	All tests
`Built-ins (tmp_path…)`	All tests

REQUEST OBJECT

ATTRIBUTE	GIVES
`request.param`	Current value of params=
`request.node`	The test item
`request.config`	Config and CLI options
`request.getfixturevalue(n)`	Another fixture by name
`request.node.get_closest_marker(m)`	A mark on the test

⚠ COMMON MISTAKES

- Importing fixtures from `conftest.py`, which registers them twice and can break overrides
- Parametrizing a fixture that many tests use and doubling the suite's run time without noticing
- A factory that never cleans up, leaving rows behind for the next test

✓ TIPS & BEST PRACTICES

- Use `pytest --fixtures-per-test path::test` to see which fixtures a test gets and from where
- Read marks with `request.node.get_closest_marker` to configure a fixture per test
- Keep fixtures next to the tests that use them; move them up only when shared

QUICK SUMMARY

- `conftest.py` — fixtures for a folder tree
- closest definition wins
- factory fixture — return a function
- `params=` + `request.param` — run per value

Built-in Fixtures

FIXTURES PYTEST GIVES YOU

pytest ships fixtures for the chores every test suite needs: a fresh temporary directory (`tmp_path`), captured stdout and stderr (`capsys`, `capfd`), captured log records (`caplog`), safe patching of attributes and environment variables (`monkeypatch`, sheet 13), and access to the test and config (`request`, `pytestconfig`). Request them like any fixture.

NOTES

- `tmp_path` is a `pathlib.Path`, unique per test, kept for the last 3 runs
- `capsys.readouterr()` returns `(out, err)` and resets the buffers
- `caplog.set_level(logging.DEBUG)` captures lower levels
- `tmpdir` (py.path) is legacy; use `tmp_path`

USING THE BUILT-INS

```
import logging, json

def test_writes_report(tmp_path):
    out = tmp_path / "report.json"
    write_report(out, {"total": 3})
    assert json.loads(out.read_text()) == {"total": 3}

def test_cli_prints(capsys):
    main(["--version"])
    out, err = capsys.readouterr()
    assert out.strip() == "orders 1.4.0"
    assert err == ""

def test_warns_in_log(caplog):
    caplog.set_level(logging.WARNING, logger="shop")
    apply_discount(cart, 0.9)
    assert "large discount" in caplog.text
    assert caplog.records[0].levelname == "WARNING"

def test_reads_env(monkeypatch):
    monkeypatch.setenv("APP_ENV", "test")
    assert load_settings().env == "test"

def test_option(pytestconfig):
    if pytestconfig.getoption("--runslow"): ...
```

CODE EXPLAINED

→ Each test gets its own empty `tmp_path`, so file tests never collide

→ `readouterr()` empties the buffer, so call it again after more output

→ `caplog.text` is the formatted log; `caplog.records` are `LogRecord` objects

→ `pytestconfig.getoption` reads custom CLI options added in `conftest.py`

UNITTEST EQUIVALENT

≈ `tmp_path` → `tempfile.TemporaryDirectory()` in `setUp` + `addCleanup`

≈ `capsys` → `contextlib.redirect_stdout(io.StringIO())`

≈ `caplog` → `self.assertLogs("shop", level="WARNING")`

≈ `monkeypatch.setenv` → `unittest.mock.patch.dict(os.environ, …)`

PREFER / AVOID

⇄ `tempfile.mkdtemp()` with manual cleanup → `tmp_path`

⇄ `tmpdir` → `tmp_path`

⇄ patching `sys.stdout` by hand → `capsys`

BUILT-IN FIXTURES

FIXTURE	GIVES
`tmp_path`	Fresh pathlib.Path per test
`tmp_path_factory`	Make dirs in wider scopes
`capsys`	Captured sys.stdout / stderr
`capfd`	Captured file descriptors 1 and 2 (subprocesses, C code)
`caplog`	Log records and text
`monkeypatch`	Undoable patches
`recwarn`	Recorded warnings
`request / pytestconfig`	Test and config access

CAPLOG API

CALL	DOES
`caplog.set_level(lvl, logger=)`	Capture from this level
`caplog.at_level(lvl)`	Context manager version
`caplog.text`	Formatted output
`caplog.records`	LogRecord list
`caplog.messages`	Message strings
`caplog.clear()`	Reset

⚠ COMMON MISTAKES

- Writing test files into the repo or `/tmp` instead of `tmp_path`
- Using `capsys` for output from a subprocess or C extension; that needs `capfd`
- Expecting `caplog` to catch DEBUG messages without setting the level

✓ TIPS & BEST PRACTICES

- Use `tmp_path_factory.mktemp` in session fixtures that need a directory
- Assert on `caplog.messages` for exact messages without timestamps
- Run `pytest --fixtures` to see every built-in and plugin fixture available

QUICK SUMMARY

- `tmp_path` — temp directory
- `capsys` / `capfd` — captured output
- `caplog` — captured logs
- `monkeypatch` / `pytestconfig` / `request` — patching and context

Parametrize

ONE TEST, MANY CASES

@pytest.mark.parametrize runs a test once for each set of arguments, and each run is reported as its own test with its own ID. It replaces copy-pasted tests and `for` loops that stop at the first failure. Stack decorators to test every combination, and wrap a case in `pytest.param` to give it an ID or a mark.

NOTES

- Argument names go in one comma-separated string
- IDs appear in brackets: `test_total[empty]`
- Select one case with `-k empty` or its node ID
- `indirect=True` sends values to a fixture instead of the test

UNDER THE HOOD

- parametrize expands one test function into several **items** during collection
- IDs are built from the values (numbers, strings) or `argN`, unless `ids=` names them
- `strict_parametrization_ids` (in `strict`) rejects duplicate IDs

CASES IN A TABLE

```python
import pytest

@pytest.mark.parametrize(
    "prices, discount, expected",
    [
        ([], 0, 0),
        ([10, 10], 0.25, 15.0),
        ([9.99], 0, 9.99),
        pytest.param([1], 0.999, 0.0, id="rounds-to-zero"),
        pytest.param([1], 1.5, None, marks=pytest.mark.xfail(raises=ValueError)),
    ],
    ids=["empty", "quarter-off", "one-item", None, None],
)
def test_total(prices, discount, expected):
    assert total(prices, discount) == expected

@pytest.mark.parametrize("currency", ["USD", "EUR"])
@pytest.mark.parametrize("country", ["US", "DE", "JP"])
def test_tax(country, currency):     # 6 combinations
    assert tax_rate(country, currency) >= 0

@pytest.mark.parametrize("db", ["sqlite"], indirect=True)
def test_on_sqlite(db): ...
```

CODE EXPLAINED

→ Each tuple is one case; its values map to the names in order

→ `pytest.param(..., id=)` names one case; `None` in `ids` keeps the generated ID

→ Stacked decorators multiply: 3 countries × 2 currencies = 6 tests

→ `indirect=True` passes `"sqlite"` to the `db` fixture as `request.param`

UNITTEST EQUIVALENT

≈ `parametrize` → a loop with `with self.subTest(case=c):` inside one test

≈ each case reported separately → `subTest` reports failures per case but counts one test

≈ stacked decorators → nested loops of `subTest`

PREFER / AVOID

⇄ a `for` loop of asserts → `parametrize`

⇄ unreadable IDs like `test_x[case3]` → `ids=` or `pytest.param(id=)`

⇄ copy-pasted tests differing by one value → one parametrized test

PARAMETRIZE FORMS

FORM	EXAMPLE
`One arg`	parametrize("n", [1, 2, 3])
`Several args`	parametrize("a, b", [(1, 2), (3, 4)])
`Named case`	pytest.param(..., id="empty")
`Marked case`	pytest.param(..., marks=pytest.mark.slow)
`Indirect`	parametrize("db", [...], indirect=True)
`On a class`	Applies to every method

RUNNING CASES

COMMAND	RUNS
`pytest -k empty`	Cases whose ID contains "empty"
`pytest "tests/test_cart.py::test_total[empty]"`	One case
`pytest --co -q`	List all generated IDs

⚠ COMMON MISTAKES

- Passing argument names as a list of one string with commas missing: `"prices discount"`
- Building huge cartesian products with stacked decorators that take minutes
- Mutable defaults in case data (a shared list) that one case modifies

✓ TIPS & BEST PRACTICES

- Put the expected value in each case so the table reads as a spec
- Give cases IDs that describe the situation, not the values
- Quote node IDs with brackets in the shell: `"…::test_total[empty]"`

QUICK SUMMARY

- `parametrize("a, b", [...])` — one test per case
- `ids=` / `pytest.param(id=)` — readable names
- stacked — every combination
- `indirect=True` — send values to a fixture

Subtests

SEVERAL CHECKS, EACH REPORTED

pytest 9 adds a built-in **subtests** fixture (it used to be the pytest-subtests plugin). Inside one test, each `with subtests.test(...)` block is reported separately, and a failure in one block doesn't stop the others. Use it when the cases are only known at run time (rows from a file, items from an API); use **parametrize** when they're known when the test is collected.

NOTES

- Keyword arguments to `subtests.test` label the block in the report
- A failing subtest marks the whole test as failed
- `unittest.TestCase.subTest` blocks are now reported the same way
- Fixtures run once for the test, not once per subtest

SUBTESTS IN PRACTICE

```python
import json

def test_every_fixture_file_loads(subtests, tmp_path):
    for path in sorted(Path("tests/data").glob("*.json")):
        with subtests.test(msg="loads", file=path.name):
            data = json.loads(path.read_text())
            assert "id" in data

def test_all_routes_require_auth(subtests, client):
    for route in app.routes():                # known only at run time
        with subtests.test(route=route.path):
            assert client.get(route.path).status_code == 401

# Output (abridged):
# test_api.py::test_all_routes_require_auth [route='/admin'] SUBFAIL
# test_api.py::test_all_routes_require_auth [route='/orders'] SUBPASS
# test_api.py::test_all_routes_require_auth FAILED

# unittest style, also reported per block in pytest 9:
class TestFiles(unittest.TestCase):
    def test_load(self):
        for name in names:
            with self.subTest(name=name):
                self.assertTrue(load(name))
```

CODE EXPLAINED

- → Each block's label comes from `msg` and the keyword arguments
- → The loop keeps going after a failing block, so you see every bad route at once
- → The parent test is reported as failed if any subtest failed
- → Expensive fixtures (like `client`) are built once, not per subtest

UNITTEST EQUIVALENT

- ≈ `subtests.test(route=…)` → `self.subTest(route=…)` (same idea, now reported per block by pytest)
- ≈ pytest runs existing `subTest` code without changes

PREFER / AVOID

- ⇄ a `for` loop with bare asserts → `subtests.test` blocks
- ⇄ subtests for a fixed list known up front → `parametrize`
- ⇄ dozens of subtests per test → split the test; reports get hard to read

SUBTESTS VS PARAMETRIZE

QUESTION	PARAMETRIZE	SUBTESTS
Cases known at collection?	Required	Not needed
Each case a separate test ID?	Yes	No (one test)
Run one case alone?	Yes (-k / node ID)	No
Fixtures rebuilt per case?	Yes (function scope)	No
Shown in --co?	Yes	No

REPORT WORDS

WORD	MEANS
SUBPASS	Block passed
SUBFAIL	Block failed
SUBSKIP	Block skipped
FAILED (parent)	At least one block failed

⚠ COMMON MISTAKES

- Using subtests where parametrize fits and losing the ability to rerun one case
- Expecting a fresh fixture per subtest; state carries over between blocks
- Installing the old pytest-subtests plugin alongside pytest 9

✓ TIPS & BEST PRACTICES

- Put the identifying value in the keyword arguments so the failing case is obvious
- Use subtests for data-driven checks over files, routes or database rows
- Remove `pytest-subtests` from dependencies when moving to pytest 9

QUICK SUMMARY

- `subtests.test(**labels)` — report each block
- keeps going after a failure
- cases at run time → subtests
- cases at collection → parametrize

Marks: skip, xfail & Custom

LABELS THAT CHANGE HOW TESTS RUN

A **mark** is a decorator that attaches metadata to a test. Built-in marks **skip** a test always or on a condition, or mark it **xfail** (expected to fail) so a known bug doesn't break CI. **Custom marks** like `slow` or `integration` let you select or exclude groups with `-m`. Register every custom mark in config so typos are caught.

NOTES

- `-ra` in `addopts` lists the reasons for skips and xfails at the end
- `xfail(strict=True)` fails if the test unexpectedly passes
- Marks on a class or with `pytestmark = …` apply to every test inside
- Call `pytest.skip("reason")` inside a test to skip at run time

SKIPPING, XFAIL AND SELECTING

```
import sys, pytest

@pytest.mark.skip(reason="payment sandbox is down")
def test_charge(): ...

@pytest.mark.skipif(sys.platform == "win32", reason="POSIX paths only")
def test_symlinks(tmp_path): ...

@pytest.mark.xfail(reason="bug #142: rounding", strict=True, raises=AssertionError)
def test_rounding():
    assert total([0.1, 0.2]) == 0.3

np = pytest.importorskip("numpy")      # skip the module if missing

@pytest.mark.slow
@pytest.mark.integration
def test_full_checkout(db): ...

pytestmark = pytest.mark.integration    # every test in this module

# pyproject.toml
# [tool.pytest]
# markers = ["slow: >1s", "integration: needs the database"]
# $ pytest -m "not slow"
# $ pytest -m "integration and not slow"
```

CODE EXPLAINED

- → `skipif` takes any boolean expression evaluated at collection
- → `strict=True` makes a fixed bug show up as a failure, reminding you to remove the mark
- → `importorskip` returns the module when it's installed
- → `-m` takes a boolean expression of mark names

UNITTEST EQUIVALENT

- ≈ `@pytest.mark.skip` → `@unittest.skip("reason")`
- ≈ `skipif` → `@unittest.skipIf(cond, "reason")`
- ≈ `xfail` → `@unittest.expectedFailure`
- ≈ custom marks + `-m` → no equivalent; split test files or use name patterns

PREFER / AVOID

- ⇄ commenting out a broken test → `xfail(strict=True)` with the issue number
- ⇄ unregistered marks → register under `markers` and use `strict`
- ⇄ `skip` with no reason → always give a reason

BUILT-IN MARKS

MARK	EFFECT
`skip(reason=)`	Never run
`skipif(cond, reason=)`	Skip when true
`xfail(reason=, strict=, raises=)`	Expected to fail
`parametrize`	Run per case (sheet 10)
`usefixtures("x")`	Apply fixtures
`filterwarnings("ignore:…")`	Per-test warning filter

-M EXPRESSIONS

EXPRESSION	SELECTS
`-m slow`	Tests marked slow
`-m "not slow"`	Everything else
`-m "slow or integration"`	Either
`-m "integration and not slow"`	Combination
`--markers`	List registered marks

⚠ COMMON MISTAKES

- Leaving `xfail` without `strict`, so a fixed bug quietly turns into an XPASS nobody notices
- Typos like `@pytest.mark.integation` silently creating a new mark when not using `strict`
- Skipping on a condition evaluated at import time that depends on test setup

✓ TIPS & BEST PRACTICES

- Always add a reason and an issue link to `skip` and `xfail`
- Run fast tests locally with `-m "not slow"` and everything in CI
- Use `-ra` to review skips regularly; skipped tests rot

QUICK SUMMARY

- `skip` / `skipif` — don't run
- `xfail(strict=True)` — known failure
- custom marks + `-m` — select groups
- register marks — catch typos

monkeypatch

`setattr` · `setenv` / `delenv` · `setitem` · `chdir` · `syspath_prepend` · `context()` · automatic undo

TEMPORARY PATCHES, UNDONE FOR YOU

The **`monkeypatch`** fixture replaces attributes, dictionary items, environment variables and the working directory for the length of one test, then puts everything back during teardown. Use it to swap a slow or external function for a fake, set environment variables, or simulate a missing key, without leaking changes into other tests.

NOTES

- Patch the name **where it is looked up**: `"shop.cart.requests.get"`, not `"requests.get"`
- `raising=False` allows setting an attribute that doesn't exist yet
- `monkeypatch.context()` limits a patch to a `with` block
- It's function-scoped; use `pytest.MonkeyPatch.context()` in wider fixtures

UNDER THE HOOD

- Each patch records the original value on an undo stack
- At teardown, `undo()` restores entries in reverse order
- That's why it can't be requested by module- or session-scoped fixtures

PATCHING WITHOUT CLEANUP CODE

```python
import pytest
import shop.rates

def test_converts_with_fake_rate(monkeypatch):
    monkeypatch.setattr(shop.rates, "fetch_rate", lambda cur: 2.0)
    assert convert(10, "EUR") == 20.0

def test_dotted_string_target(monkeypatch):
    monkeypatch.setattr("shop.cart.now", lambda: 1_700_000_000)

def test_env(monkeypatch):
    monkeypatch.setenv("API_URL", "http://localhost:9999")
    monkeypatch.delenv("API_TOKEN", raising=False)
    assert Settings().api_url.endswith("9999")

def test_dict_item(monkeypatch):
    monkeypatch.setitem(CONFIG, "retries", 0)
    monkeypatch.delitem(CONFIG, "proxy", raising=False)

def test_cwd(monkeypatch, tmp_path):
    monkeypatch.chdir(tmp_path)
    write_default_config()
    assert (tmp_path / "config.toml").exists()

@pytest.fixture(scope="module")
def offline():
    with pytest.MonkeyPatch.context() as mp:
        mp.setattr("socket.socket", None)
        yield
```

CODE EXPLAINED

- → `setattr(module, "name", value)` or the dotted-string form both work
- → `delenv(..., raising=False)` doesn't fail if the variable isn't set
- → `chdir` is undone too, so relative paths in later tests still work
- → `pytest.MonkeyPatch.context()` is the way to patch in wider-scoped fixtures

UNITTEST EQUIVALENT

- ≈ `monkeypatch.setattr` → `unittest.mock.patch("mod.name", value)` as decorator or context manager
- ≈ `setenv` → `patch.dict(os.environ, {"API_URL": …})`
- ≈ `setitem` → `patch.dict(CONFIG, {…})`
- ≈ automatic undo → `patch` restores when the block exits

PREFER / AVOID

- ⇄ assigning `module.func = fake` by hand → `monkeypatch.setattr`
- ⇄ patching `requests.get` globally → patch `shop.cart.requests.get` where it's used
- ⇄ `os.environ["X"] = "y"` in a test → `monkeypatch.setenv`

MONKEYPATCH API

METHOD	PATCHES
`setattr(obj, name, value)`	Attribute
`delattr(obj, name)`	Remove attribute
`setitem(d, key, value)`	Dict item
`delitem(d, key)`	Remove item
`setenv(name, value)`	Environment variable
`delenv(name)`	Unset variable
`chdir(path)`	Working directory
`syspath_prepend(path)`	sys.path

MONKEYPATCH VS MOCK

NEED	USE
Swap a value or function	monkeypatch.setattr
Set / unset env vars	monkeypatch.setenv / delenv
Check how it was called	mocker.patch / Mock (sheet 14)
Return different values per call	Mock side_effect
Keep real behavior, record calls	mocker.spy

⚠ COMMON MISTAKES

- Patching the original module after the code under test did `from x import fn`
- Setting `os.environ` directly and leaking the value into later tests
- Requesting `monkeypatch` from a module-scoped fixture (ScopeMismatch)

✓ TIPS & BEST PRACTICES

- Find the right target by asking "which module's namespace does the code read this name from?"
- Use monkeypatch for simple replacements; use mocks (sheet 14) when you need to check calls
- Combine with `tmp_path` and `chdir` to test code that uses relative paths

QUICK SUMMARY

- `setattr` — swap a function or value
- `setenv` / `delenv` — environment
- `chdir` — working directory
- undone automatically at teardown

unittest.mock & pytest-mock

`mocker.patch` · `MagicMock` · `return_value` · `side_effect` · `assert_called_once_with` · `mocker.spy` · `autospec=True`

FAKES THAT RECORD HOW THEY WERE USED

A **mock** stands in for a real object and records every call, so a test can check *how* the code used it. The standard library's **`unittest.mock`** provides `Mock`, `MagicMock` and `patch`; the **`pytest-mock`** plugin wraps them in a **`mocker`** fixture that undoes patches automatically. Configure results with `return_value` and `side_effect`, then assert on calls.

NOTES

- `pip install pytest-mock` for the `mocker` fixture
- `autospec=True` makes the mock reject calls that don't match the real signature
- `mocker.spy` wraps a real function: it still runs, but calls are recorded
- The same "patch where it's looked up" rule applies

UNDER THE HOOD

- `patch` replaces the name in the target module's namespace for the duration, then restores it
- `autospec` builds the mock from the real object, so wrong argument counts and missing attributes raise
- A plain `MagicMock` accepts anything, which can hide bugs

MOCKING AN HTTP CLIENT

```python
def test_fetch_rate(mocker):
    get = mocker.patch("shop.rates.httpx.get", autospec=True)
    get.return_value.json.return_value = {"rate": 1.1}

    assert fetch_rate("EUR") == 1.1
    get.assert_called_once_with("https://rates.example/EUR", timeout=5)

def test_retries_then_succeeds(mocker):
    send = mocker.patch("shop.mail.smtp_send", autospec=True)
    send.side_effect = [ConnectionError, ConnectionError, None]
    notify("ann@example.com")
    assert send.call_count == 3

def test_raises(mocker):
    mocker.patch("shop.rates.fetch_rate", side_effect=TimeoutError)
    with pytest.raises(RateUnavailable):
        convert(10, "EUR")

def test_spy(mocker):
    spy = mocker.spy(shop.cart, "recalculate")
    cart.add("apple")
    spy.assert_called_once()           # real recalculate still ran

# without the plugin
from unittest.mock import patch
def test_plain():
    with patch("shop.rates.fetch_rate", return_value=2.0) as m:
        assert convert(1, "EUR") == 2.0
```

CODE EXPLAINED

- → `return_value.json.return_value` configures what `httpx.get(...).json()` returns
- → `side_effect` as a list returns or raises each item in turn
- → `side_effect=TimeoutError` makes every call raise
- → `mocker.spy` keeps the real behavior and adds call recording

UNITTEST EQUIVALENT

- ≈ `mocker.patch(...)` → `@patch("…")` decorator or `with patch(…)` block
- ≈ `mocker.spy` → `patch.object(obj, "fn", wraps=obj.fn)`
- ≈ `mocker` auto-undo → `patch` restores on exit, or `self.addCleanup(p.stop)`

PREFER / AVOID

- ⇄ bare `MagicMock()` for a real class → `autospec=True` / `create_autospec`
- ⇄ asserting every internal call → assert on results; check calls only at boundaries
- ⇄ `mock.called == True` → `mock.assert_called_once_with(…)`

CONFIGURING A MOCK

ATTRIBUTE	EFFECT
`return_value`	What a call returns
`side_effect = Exc`	Every call raises
`side_effect = [a, b]`	Returns / raises in turn
`side_effect = fn`	Calls fn with the same args
`spec / autospec`	Restrict to the real API

CALL ASSERTIONS

METHOD	CHECKS
`assert_called()`	Called at least once
`assert_called_once_with(…)`	Exactly once, these args
`assert_any_call(…)`	Some call matched
`assert_not_called()`	Never called
`call_count`	Number of calls
`call_args_list`	Every call's args

⚠ COMMON MISTAKES

- Typos like `assert_caled_once()` that silently passed on old Python; current versions raise `AttributeError`
- Mocking the code under test itself, so the test checks only the mock
- Patching `httpx.get` instead of `shop.rates.httpx.get` and making real requests

✓ TIPS & BEST PRACTICES

- Use `autospec=True` by default; it catches signature drift when the real API changes
- Mock at the boundary (network, clock, filesystem), not your own internal functions
- Prefer a simple fake class over a deeply configured `MagicMock` chain

QUICK SUMMARY

- `mocker.patch(target)` — patch + auto-undo
- `return_value` / `side_effect` — behavior
- `assert_called_once_with` — check usage
- `autospec=True` — match the real signature

Selecting & Rerunning Tests

RUN EXACTLY THE TESTS YOU NEED

Running the whole suite for every change is slow. Select tests by **path or node ID**, by **name** (`-k`), or by **mark** (`-m`). After a failure, rerun only what failed with **`--lf`** (last failed), put failures first with **`--ff`**, stop at the first failure with **`-x`**, or step through failures one at a time with **`--sw`** (stepwise).

NOTES

- Results of the last run live in `.pytest_cache/` (gitignore it)
- `-k` matches substrings of test, class and file names
- Combine selectors: `pytest tests/api -k refund -m "not slow"`
- `--co` with any selector shows what would run

SELECTION RECIPES

```
pytest tests/test_cart.py                    # one file
pytest tests/test_cart.py::test_total        # one test
pytest tests/test_cart.py::TestDiscount      # one class
pytest "tests/test_cart.py::test_total[empty]"   # one case
pytest tests/api/                            # one folder
pytest -k "refund and not partial"           # by name expression
pytest -m "not slow"                         # by mark
pytest --lf                                  # only last failures
pytest --ff                                  # failures first, then the rest
pytest -x                                    # stop at first failure
pytest --maxfail=3                           # stop after 3
pytest --sw                                  # stepwise: resume from last failure
pytest --co -q -k refund                     # list matches without running
pytest --deselect tests/test_api.py::test_flaky
```

CODE EXPLAINED

→ Node IDs are printed in failure output; copy one to rerun that test

→ `--lf` with nothing failed last time runs everything (or nothing with `--lfnf=none`)

→ `--sw` runs until a failure, then next time starts from that test

→ `--deselect` removes specific node IDs from a run

UNITTEST EQUIVALENT

≈ `pytest file::test` → `python -m unittest tests.test_cart.TestCart.test_total`

≈ `-k` → `python -m unittest -k pattern`

≈ `-x` → `python -m unittest -f` (failfast)

≈ `--lf` / `--ff` → no equivalent

PREFER / AVOID

⇄ commenting out tests to run one → a node ID or `-k`

⇄ rerunning the full suite after one failure → `--lf`

⇄ scrolling for the failing test name → copy the node ID from the short summary

SELECTION FLAGS

FLAG	DOES
`path::name`	Node ID
`-k EXPR`	Name expression (and / or / not)
`-m EXPR`	Mark expression
`--lf`	Last failed only
`--ff`	Failed first
`--nf`	New tests first
`-x / --maxfail=N`	Stop early
`--sw / --sw-skip`	Stepwise

-K EXAMPLES

EXPRESSION	MATCHES
`-k cart`	Any name containing "cart"
`-k "not slow"`	Names without "slow"
`-k "TestDiscount and quarter"`	Class + method
`-k "empty or zero"`	Either word

⚠ COMMON MISTAKES

- Committing `.pytest_cache/` to Git
- Forgetting quotes around node IDs with brackets, so the shell expands them
- Leaving `-x` in `addopts`, so CI reports only the first failure

✓ TIPS & BEST PRACTICES

- Use `--lf -x` while fixing failures one by one
- Use `--sw` to walk through a broken suite after a big refactor
- Add `.pytest_cache/` to `.gitignore`

QUICK SUMMARY

- node ID — `file::Class::test[case]`
- `-k` / `-m` — by name / by mark
- `--lf` / `--ff` — rerun failures
- `-x` / `--sw` — stop or step

Output & Debugging

READ FAILURES AND DROP INTO A DEBUGGER

pytest captures `print` output and shows it only for failing tests. Control how much you see with `-v` / `-q`, the traceback style with `--tb`, and the end-of-run summary with `-r`. When a failure isn't obvious, **`--pdb`** opens the debugger at the failing line, and a **`breakpoint()`** call in a test or in your code stops there with capture turned off.

NOTES

- `-s` turns capture off so prints appear live
- `--pdb` with `-x` stops at the first failure and debugs it
- `--durations=10` lists the slowest tests and fixtures
- `log_cli = true` streams log records as tests run

UNDER THE HOOD

- Capture works at the **file-descriptor** level by default (`--capture=fd`), catching output from C code too
- Captured output is attached to the report and printed only for failures
- `breakpoint()` and `--pdb` suspend capture so the debugger can talk to the terminal

DEBUGGING A FAILURE

```
pytest -v                       # one line per test
pytest -q                       # quieter
pytest -s                       # show print() output live
pytest -rA                      # summary of all outcomes at the end
pytest -rfE                     # summary of failures and errors only
pytest --tb=short               # shorter tracebacks (also line / no / native)
pytest -l                       # show local variables in tracebacks
pytest -x --pdb                 # debugger at the first failure
pytest --trace tests/test_cart.py::test_total   # debugger at test start
pytest --durations=10           # slowest 10 setups / calls / teardowns
pytest -o log_cli=true --log-cli-level=DEBUG

# in code or a test:
def test_total():
    cart = build_cart()
    breakpoint()                # (Pdb) p cart.items   n   s   c   q
```

CODE EXPLAINED

→ `-r` takes letters: `f` failed, `E` error, `s` skipped, `x` xfailed, `X` xpassed, `p` passed, `A` all

→ `--tb=short` is usually enough; `long` (default) shows full context

→ `-o` overrides any config key from the command line

→ In pdb: `p expr` print, `n` next, `s` step in, `c` continue, `q` quit

UNITTEST EQUIVALENT

≈ `-v` → `python -m unittest -v`

≈ `--pdb` → no flag; run under `python -m pdb` or add `breakpoint()`

≈ `-s` → unittest doesn't capture by default; `-b` buffers output

PREFER / AVOID

⇄ adding prints and rerunning → `--pdb` or `breakpoint()`

⇄ reading full tracebacks every time → `--tb=short` in `addopts`

⇄ guessing why the suite is slow → `--durations=10`

--TB STYLES

STYLE	SHOWS
`auto / long`	Full context (default)
`short`	Shorter, one frame per line of context
`line`	One line per failure
`native`	Standard Python traceback
`no`	No traceback

PDB COMMANDS

COMMAND	DOES
`p expr / pp expr`	Print / pretty-print
`n`	Next line
`s`	Step into
`c`	Continue
`l / ll`	List code
`w`	Where (stack)
`interact`	Python REPL in this frame

⚠ COMMON MISTAKES

- Leaving `breakpoint()` in committed code; CI hangs or errors waiting for input
- Using `-s` all the time and drowning failures in output
- Reading only the last line of a traceback and missing the real cause above it

✓ TIPS & BEST PRACTICES

- Set `PYTHONBREAKPOINT=0` in CI to make stray `breakpoint()` calls no-ops
- Use `--pdbcls=IPython.terminal.debugger:TerminalPdb` for a nicer debugger
- Add `--durations=10` to CI to catch tests getting slower

QUICK SUMMARY

- `-v` / `-q` / `-s` — verbosity and capture
- `-r` / `--tb` — summary and traceback style
- `--pdb` / `breakpoint()` — debug
- `--durations` — find slow tests

Plugins & Hooks

EXTEND PYTEST FROM CONFTEST.PY

pytest is built from **plugins**, and so are its extensions: install a package such as pytest-cov and it registers itself. Your own `conftest.py` is a local plugin too. It can implement **hooks**, functions with fixed `pytest_*` names that pytest calls at points in a run, such as adding a CLI option, changing collected tests, or reporting results.

NOTES

- `pytest --trace-config` lists active plugins
- `-p no:name` disables a plugin for one run
- All hooks are listed in the "API Reference → Hooks" docs
- Hook arguments are matched by name; take only what you need

UNDER THE HOOD

- Hooks are called through **pluggy**: every registered implementation runs, in plugin order
- `firstresult` hooks stop at the first non-`None` return
- `@pytest.hookimpl(wrapper=True)` wraps other implementations with `yield`

A --RUNSLOW OPTION IN CONFTEST.PY

```python
import pytest

def pytest_addoption(parser):
    parser.addoption("--runslow", action="store_true",
                     help="run tests marked slow")

def pytest_configure(config):
    config.addinivalue_line("markers", "slow: takes more than a second")

def pytest_collection_modifyitems(config, items):
    if config.getoption("--runslow"):
        return
    skip_slow = pytest.mark.skip(reason="needs --runslow")
    for item in items:
        if "slow" in item.keywords:
            item.add_marker(skip_slow)

@pytest.hookimpl(wrapper=True)
def pytest_runtest_call(item):
    start = time.perf_counter()
    result = yield                          # run the test
    item.user_properties.append(("secs", time.perf_counter() - start))
    return result

def pytest_report_header(config):
    return f"api: {os.environ.get('API_URL', 'local')}"
```

CODE EXPLAINED

- → `pytest_addoption` adds `--runslow`; fixtures read it with `pytestconfig.getoption`
- → `pytest_collection_modifyitems` can skip, reorder or deselect tests after collection
- → The wrapper hook runs code around every test call
- → `pytest_report_header` adds a line to the top of the output

UNITTEST EQUIVALENT

- ≈ plugins and hooks → no plugin system; subclass `TestRunner` / `TestResult` or use `load_tests`
- ≈ `pytest_collection_modifyitems` → a custom `load_tests(loader, tests, pattern)` function

PREFER / AVOID

- ⇄ editing every slow test to check an env var → one `collection_modifyitems` hook
- ⇄ copy-pasting helpers between repos → package them as a plugin with an entry point
- ⇄ old `hookwrapper=True` + `outcome.get_result()` → `wrapper=True` + `result = yield`

COMMON HOOKS

HOOK	CALLED
`pytest_addoption(parser)`	Start-up: add CLI / config options
`pytest_configure(config)`	After options are parsed
`pytest_collection_modifyitems(config, items)`	After collection
`pytest_runtest_setup(item)`	Before each test's setup
`pytest_runtest_makereport(item, call)`	Building each report
`pytest_sessionfinish(session, exitstatus)`	End of run
`pytest_report_header(config)`	Header lines

PLUGIN COMMANDS

COMMAND	DOES
`pip install pytest-xyz`	Auto-registers via entry point
`-p no:cacheprovider`	Disable a plugin
`-p mymodule`	Load a module as a plugin
`--trace-config`	Show loaded plugins
`pytest_plugins = ["x"]`	Load from root conftest.py

⚠ COMMON MISTAKES

- Defining `pytest_plugins` in a non-root `conftest.py`, which is an error
- Misspelling a hook name; pytest only warns about unknown `pytest_*` names in strict setups
- Doing heavy work in `pytest_configure` that slows every run, even `--co`

✓ TIPS & BEST PRACTICES

- Start with `conftest.py` hooks; turn them into a package only when several repos need them
- Use `item.user_properties` to attach data that shows up in JUnit XML
- Read a plugin's source; most are a few hundred lines of hooks

QUICK SUMMARY

- plugins — installed packages and `conftest.py`
- hooks — `pytest_*` functions pytest calls
- `addoption` + `collection_modifyitems` — custom flags
- `wrapper=True` — run code around a hook

Coverage with pytest-cov

`--cov=pkg` · `--cov-report=term-missing` / `html` · `--cov-branch` · `--cov-fail-under` · `[tool.coverage]` · excluding lines

WHICH LINES THE TESTS ACTUALLY RAN

pytest-cov runs coverage.py during the test run and reports which lines (and branches) your tests executed. `--cov=package` picks what to measure; `--cov-report` picks the output; `--cov-fail-under` fails the run below a threshold. Coverage shows untested code; it doesn't prove tested code is correct.

NOTES

- `pip install pytest-cov`
- Branch coverage catches `if` statements whose `else` path never ran
- Settings go in `[tool.coverage.*]` in `pyproject.toml`
- Works with xdist (`-n`) and combines results automatically

MEASURING COVERAGE

```
pytest --cov=orders                            # terminal summary
pytest --cov=orders --cov-report=term-missing # + missing line numbers
pytest --cov=orders --cov-report=html         # htmlcov/index.html
pytest --cov=orders --cov-report=xml          # coverage.xml for CI tools
pytest --cov=orders --cov-branch --cov-fail-under=90

# Name                  Stmts   Miss Branch BrPart  Cover   Missing
# src/orders/cart.py       42      3     14      2    91%   55-57, 61->64
# TOTAL                   310     18     96      9    93%

# pyproject.toml
[tool.coverage.run]
branch = true
source = ["orders"]
omit = ["*/migrations/*"]

[tool.coverage.report]
fail_under = 90
show_missing = true
exclude_also = ["if TYPE_CHECKING:", "raise NotImplementedError"]
```

CODE EXPLAINED

- → `Missing` lists line numbers never run; `61->64` is a branch never taken
- → `--cov-branch` adds the Branch and BrPart columns
- → `exclude_also` adds patterns to skip without replacing the defaults
- → Put `--cov` in CI scripts rather than `addopts`, since it slows local runs and the debugger

UNITTEST EQUIVALENT

- ≈ pytest-cov → `coverage run -m unittest` then `coverage report -m`
- ≈ `--cov-fail-under` → `coverage report --fail-under=90`
- ≈ same coverage.py settings in both cases

PREFER / AVOID

- ⇄ chasing 100% with tests that assert nothing → meaningful tests on the risky code
- ⇄ line coverage only → `--cov-branch`
- ⇄ `# pragma: no cover` sprinkled everywhere → `exclude_also` patterns in config

REPORT TYPES

--COV-REPORT	OUTPUT
`term`	Summary table
`term-missing`	Plus missing lines
`html`	Browsable htmlcov/
`xml`	Cobertura XML (CI, Codecov)
`json / lcov`	Machine-readable

COVERAGE SETTINGS

KEY	EFFECT
`run.branch`	Measure branches
`run.source`	What to measure
`run.omit`	Files to skip
`report.fail_under`	Threshold
`report.exclude_also`	Lines to ignore
`report.skip_covered`	Hide 100% files

⚠ COMMON MISTAKES

- Measuring the tests themselves and inflating the number; set `source` to your package
- Treating a high percentage as proof of good tests
- Leaving `--cov` in `addopts` and wondering why breakpoints are slow

✓ TIPS & BEST PRACTICES

- Open the HTML report and read the red lines in risky modules first
- Use a modest `fail_under` and raise it over time
- Combine with mutation testing (Testing set) to check the tests catch bugs

QUICK SUMMARY

- `--cov=pkg` — measure
- `term-missing` / `html` — see gaps
- `--cov-branch` — branch coverage
- `--cov-fail-under` — enforce a floor

Parallel & Flaky Tests

FASTER RUNS, HONEST RESULTS

pytest-xdist runs tests in several worker processes: `-n auto` uses one per CPU core. Parallel runs expose tests that share state or depend on order, which is good. **pytest-randomly** shuffles order every run to find those dependencies, **pytest-rerunfailures** retries flaky tests while you fix them, and **pytest-repeat** runs a test many times to reproduce a flake.

NOTES

- Each xdist worker is a separate process with its own session fixtures
- `-p no:randomly` turns shuffling off for one run
- Reruns hide flakiness; treat them as a temporary measure
- Tests that write to the same file or port will collide under `-n`

UNDER THE HOOD

- xdist starts N workers; the controller hands out tests and collects reports
- `session` -scoped fixtures run once **per worker**, not once overall
- `--dist loadscope` keeps a module or class on one worker so module fixtures aren't rebuilt

PARALLEL, SHUFFLED, RETRIED

```
pip install pytest-xdist pytest-randomly pytest-rerunfailures pytest-repeat

pytest -n auto                        # one worker per core
pytest -n 4 --dist loadscope          # group by module / class
pytest -n auto --dist worksteal       # idle workers steal tests
pytest -p no:randomly                 # fixed order for this run
pytest --randomly-seed=1234           # reproduce an order
pytest --reruns 2 --reruns-delay 1    # retry failures twice
pytest --count 50 -x tests/test_cache.py::test_expiry   # hunt a flake

# per test
@pytest.mark.flaky(reruns=3)
def test_external_service(): ...

# unique resources per worker
@pytest.fixture(scope="session")
def db_name(worker_id):               # "gw0", "gw1"… or "master"
    return f"test_{worker_id}"
```

CODE EXPLAINED

→ `worker_id` (from xdist) lets each worker use its own database or port

→ pytest-randomly prints the seed at the top; reuse it with `--randomly-seed` to repeat an order

→ `--count` (pytest-repeat) runs one test many times; with `-x` it stops on the first failure

→ Reruns are reported as `R` so they're visible, not hidden

UNITTEST EQUIVALENT

≈ xdist → no built-in parallel runner in `unittest`

≈ random order → `unittest` runs alphabetically; no shuffle option

≈ reruns → no equivalent

PREFER / AVOID

⇄ shared files in `/tmp` → `tmp_path` per test

⇄ global reruns in `addopts` → `@pytest.mark.flaky` on known tests with an issue link

⇄ tests depending on order → each test builds its own state

XDIST --DIST MODES

MODE	BEHAVIOR
`load (default)`	Any test to any free worker
`loadscope`	Same module / class on one worker
`loadfile`	Same file on one worker
`loadgroup`	xdist_group mark keeps tests together
`worksteal`	Rebalances long-running workers

FLAKE CAUSES

CAUSE	FIX
`Shared global state`	Reset in fixtures
`Test order`	Find with pytest-randomly
`Time / timezone`	Freeze or inject the clock
`Network`	Mock the boundary
`Fixed ports / files`	worker_id or tmp_path
`Async timing`	Await events, don't sleep

⚠ COMMON MISTAKES

- Assuming a session fixture runs once under xdist and creating one database per worker by accident
- Adding `--reruns` globally and never fixing the flaky tests
- Debugging under `-n auto`; run the single test without xdist first

✓ TIPS & BEST PRACTICES

- Use `-n auto` in CI and plain runs locally while debugging
- Keep pytest-randomly installed; order bugs show up early instead of in CI
- Use `--count` with `-x` to reproduce an intermittent failure

QUICK SUMMARY

- `-n auto` — parallel workers
- `--dist loadscope` — keep module fixtures together
- pytest-randomly — shuffle to find order bugs
- `--reruns` / `flaky` — temporary retries

Async Tests

TESTING ASYNC DEF CODE

pytest can't run `async def` tests on its own. The **pytest-asyncio** plugin runs them on an event loop: mark a test with `@pytest.mark.asyncio`, or set `asyncio_mode = "auto"` so every async test and fixture is handled. **Async fixtures** work too. Projects using AnyIO or Trio can use AnyIO's pytest plugin with `@pytest.mark.anyio` instead.

NOTES

- `pip install pytest-asyncio` (1.x)
- By default each test gets its own event loop
- `loop_scope="module"` shares one loop across a module's tests
- Never call `asyncio.run` inside an async test

UNDER THE HOOD

- pytest-asyncio creates an event loop per **loop scope** and runs the test coroutine on it
- A module-scoped async fixture must run on a loop at least that wide, or you get "attached to a different loop" errors
- Set the default with `asyncio_default_fixture_loop_scope`

PYTEST-ASYNCIO

```
# pyproject.toml
# [tool.pytest]
# asyncio_mode = "auto"
# asyncio_default_fixture_loop_scope = "function"

import asyncio, httpx, pytest

@pytest.fixture
async def client():                          # async fixture
    async with httpx.AsyncClient(base_url="http://test",
                                 transport=httpx.ASGITransport(app=app)) as c:
        yield c

async def test_get_order(client):            # auto mode: no mark needed
    resp = await client.get("/orders/1")
    assert resp.status_code == 200

@pytest.mark.asyncio(loop_scope="module")
async def test_shared_loop(): ...

async def test_timeout():
    with pytest.raises(TimeoutError):
        async with asyncio.timeout(0.1):
            await slow_call()

# AnyIO instead of pytest-asyncio
@pytest.mark.anyio
async def test_with_anyio(): ...
```

CODE EXPLAINED

- → In `auto` mode every `async def` test and fixture is handled by pytest-asyncio
- → `ASGITransport` calls a FastAPI / Starlette app in memory, no server needed
- → `asyncio.timeout` (3.11+) makes a slow await fail instead of hanging
- → The AnyIO plugin needs an `anyio_backend` fixture to pick asyncio or trio

UNITTEST EQUIVALENT

- ≈ async test → `unittest.IsolatedAsyncioTestCase` with `async def test_x(self)`
- ≈ async fixture → `async def asyncSetUp(self)` / `asyncTearDown`

PREFER / AVOID

- ⇄ `asyncio.run(...)` inside a sync test → an `async def` test
- ⇄ `await asyncio.sleep(1)` to wait for work → await the event or task directly
- ⇄ mixing loop scopes by accident → set `asyncio_default_fixture_loop_scope`

ASYNCIO_MODE

MODE	BEHAVIOR
`strict (default)`	Only marked tests and @pytest_asyncio.fixture fixtures
`auto`	Every async test and fixture

SETTINGS & MARKS

ITEM	SETS
`asyncio_mode`	strict / auto
`asyncio_default_fixture_loop_scope`	Default loop for async fixtures
`@pytest.mark.asyncio(loop_scope=)`	Loop for one test
`@pytest_asyncio.fixture(loop_scope=)`	Loop for one fixture (strict mode)
`@pytest.mark.anyio`	AnyIO plugin instead

⚠ COMMON MISTAKES

- Writing `async def test_x` without the plugin; pytest skips or warns instead of running it
- A module-scoped async fixture with function-scoped loops ("Future attached to a different loop")
- Testing timing with real `sleep` calls that make the suite slow and flaky

✓ TIPS & BEST PRACTICES

- Use `asyncio_mode = "auto"` unless you mix asyncio and trio
- Wrap awaits that might hang in `asyncio.timeout`
- Test ASGI apps in memory with `httpx.ASGITransport`

QUICK SUMMARY

- pytest-asyncio — run `async def` tests
- `asyncio_mode = "auto"` — no marks needed
- async fixtures with `yield`
- `loop_scope` — share a loop

Django & Web Apps

TESTING WEB APPS WITH PYTEST

pytest-django sets up Django for pytest: it creates a test database, gives you fixtures such as `client`, `admin_client`, `rf` (request factory) and `settings`, and blocks database access unless a test is marked **`django_db`**. For FastAPI and Starlette, **`TestClient`** calls the app in memory. Flask has `app.test_client()`. The Django set covers the app side.

NOTES

- Point pytest at settings: `DJANGO_SETTINGS_MODULE = "mysite.settings"` in config
- Each `django_db` test runs in a transaction rolled back afterwards
- `transaction=True` for tests that need real commits
- Use factory_boy or factory fixtures for model data

PYTEST-DJANGO AND FASTAPI

```python
# pyproject.toml
# [tool.pytest]
# DJANGO_SETTINGS_MODULE = "mysite.settings"
# addopts = ["--reuse-db"]

import pytest
from orders.models import Order

@pytest.mark.django_db
def test_order_list(client, django_user_model):
    user = django_user_model.objects.create_user("ann", password="pw")
    Order.objects.create(owner=user, total=10)
    client.force_login(user)
    resp = client.get("/orders/")
    assert resp.status_code == 200
    assert "10.00" in resp.content.decode()

def test_admin_page(admin_client):
    assert admin_client.get("/admin/").status_code == 200

def test_feature_flag(settings, client, db):
    settings.NEW_CHECKOUT = True          # undone after the test
    assert client.get("/checkout/").status_code == 200

def test_query_count(django_assert_num_queries, db):
    with django_assert_num_queries(2):
        list(Order.objects.select_related("owner"))

# FastAPI
from fastapi.testclient import TestClient
def test_health():
    assert TestClient(app).get("/health").json() == {"ok": True}
```

CODE EXPLAINED

→ `django_user_model` is the project's user model, whatever it's called

→ Requesting the `db` fixture is the same as `@pytest.mark.django_db`

→ `settings` changes are reverted after the test

→ `--reuse-db` keeps the test database between runs for speed

UNITTEST EQUIVALENT

≈ pytest-django → Django's `TestCase` and `manage.py test`

≈ `client` fixture → `self.client`

≈ `django_db` → `TestCase` wraps each test in a transaction

≈ `settings` fixture → `@override_settings(...)`

PREFER / AVOID

⇄ `django.test.TestCase` classes for new tests → plain functions with fixtures

⇄ raw `Model.objects.create` with many fields in every test → factories

⇄ testing views through Selenium → `client` for logic, Playwright for real browser flows

PYTEST-DJANGO FIXTURES

FIXTURE	GIVES
`client / async_client`	Test client
`admin_client / admin_user`	Logged-in superuser
`rf`	RequestFactory
`db / transactional_db`	Database access
`settings`	Overridable settings
`django_assert_num_queries`	Query count check
`mailoutbox`	Sent emails

OTHER FRAMEWORKS

FRAMEWORK	TEST CLIENT
`FastAPI / Starlette`	TestClient(app) or httpx.ASGITransport
`Flask`	app.test_client() (pytest-flask)
`Django REST Framework`	APIClient

⚠ COMMON MISTAKES

- Forgetting `django_db` and getting "Database access not allowed"
- Using `transaction=True` everywhere and making the suite slow
- Running pytest without `DJANGO_SETTINGS_MODULE` configured

✓ TIPS & BEST PRACTICES

- Use `--reuse-db` locally and `--create-db` after migrations change
- Check N+1 problems with `django_assert_num_queries`
- Test views with `client`; test business logic without HTTP at all

QUICK SUMMARY

- pytest-django — Django fixtures and test DB
- `django_db` — allow database access
- `client` / `admin_client` / `settings` — common fixtures
- `TestClient` — FastAPI in memory

Testing Patterns in pytest

PATTERNS THAT KEEP TESTS CLEAR

Most good tests follow **Arrange-Act-Assert**: set up, do one thing, check the result. Factories build test data with sensible defaults; **golden files** and **snapshot** tests (syrupy) compare output to a stored, reviewed version; **Hypothesis** generates hundreds of inputs to check a property holds. The Testing set explains when to use each; this sheet shows the pytest way.

NOTES

- `pip install syrupy hypothesis`
- Snapshots are files you commit and review in diffs
- Hypothesis shrinks a failing input to the smallest example
- Test CLIs by calling `main(argv)` with `capsys`, not by spawning a process

FACTORIES, SNAPSHOTS, PROPERTIES, CLIS

```python
from hypothesis import given, strategies as st

def test_refund(make_order):                # Arrange-Act-Assert
    order = make_order(total=50)            # arrange
    refund(order, 20)                       # act
    assert order.balance == 30              # assert

def test_invoice_render(snapshot):          # syrupy
    assert render_invoice(sample_order()) == snapshot
# $ pytest --snapshot-update   (then review the diff)

def test_report_matches_golden(tmp_path):
    out = tmp_path / "report.csv"
    export_report(out)
    assert out.read_text() == Path("tests/golden/report.csv").read_text()

@given(st.lists(st.floats(min_value=0, max_value=1e6)))
def test_total_never_negative(prices):      # property-based
    assert total(prices) >= 0

def test_cli(capsys):
    exit_code = main(["export", "--format", "json"])
    assert exit_code == 0
    assert capsys.readouterr().out.startswith("{")
```

CODE EXPLAINED

→ Blank lines between arrange, act and assert make the structure visible

→ syrupy stores snapshots under `__snapshots__/`; `--snapshot-update` rewrites them

→ A golden file is a hand-checked expected output kept in the repo

→ `@given` runs the test with many generated lists; failures show the minimal example

UNITTEST EQUIVALENT

≈ factories → helper methods on a base `TestCase`

≈ syrupy → no built-in snapshot support

≈ Hypothesis → works with `unittest` too (`@given` on test methods)

PREFER / AVOID

⇄ updating snapshots without reading the diff → review every snapshot change

⇄ many near-identical example tests → one Hypothesis property

⇄ spawning `subprocess` for every CLI test → call `main(argv)` directly

PATTERN PICKER

NEED	PATTERN
`Build objects with defaults`	Factory fixture / factory_boy
`Check large output`	Snapshot (syrupy) or golden file
`Many inputs, one rule`	Hypothesis property
`Several fixed cases`	parametrize
`External service`	Fake or mock at the boundary

HYPOTHESIS STRATEGIES

STRATEGY	GENERATES
`st.integers(min_value=0)`	Ints
`st.text(max_size=20)`	Strings
`st.lists(x)`	Lists of x
`st.builds(Order, total=…)`	Objects
`st.sampled_from([…])`	One of the values
`st.one_of(a, b)`	Either strategy

⚠ COMMON MISTAKES

- Snapshot tests on output with timestamps or IDs that change every run
- Hypothesis tests that just re-implement the function to compute the expected value
- Arrange sections so long the test's point is lost

✓ TIPS & BEST PRACTICES

- Strip or freeze volatile fields (dates, UUIDs) before snapshotting
- Look for properties like round-trips (`decode(encode(x)) == x`) and invariants
- Keep each test's Act to a single call

QUICK SUMMARY

- Arrange-Act-Assert — one action per test
- syrupy / golden files — compare stored output
- Hypothesis `@given` — property-based
- `main(argv)` + `capsys` — test CLIs

pytest in CI

RUN THE SUITE ON EVERY PUSH

In CI, pytest's **exit code** decides pass or fail, and **`--junitxml`** writes a report most CI systems display. A **matrix** runs the suite on several Python versions; **tox** or **nox** run the same environments locally. Turn on strict options and warnings-as-errors in CI so problems fail early.

.GITHUB/WORKFLOWS/TEST.YML

```yaml
name: tests
on: [push, pull_request]
jobs:
  pytest:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python: ["3.12", "3.13", "3.14"]
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: ${{ matrix.python }}
      - run: python -m pip install -e . --group dev
      - run: >
          pytest -n auto -ra
          --cov=orders --cov-report=xml
          --junitxml=reports/junit.xml
          -W error --strict-markers
      - uses: actions/upload-artifact@v7
        if: always()
        with:
          name: reports-${{ matrix.python }}
          path: reports/
```

NOTES

- Exit code 5 (no tests collected) fails CI; catch it if a job may have nothing to run
- `-p no:cacheprovider` avoids writing `.pytest_cache` in read-only CI
- `CI=true` is set by most CI systems; pytest shortens some output when it sees it
- The GitHub set covers Actions syntax

CODE EXPLAINED

→ The matrix runs the same steps once per Python version

→ `if: always()` uploads reports even when tests fail

→ `-W error` turns warnings into failures only in CI

→ `pip install --group dev` (pip 25.1+) installs the `[dependency-groups] dev` packages

UNITTEST EQUIVALENT

≈ `--junitxml` → no built-in XML output; use `unittest-xml-reporting` or run unittest tests under pytest

≈ matrix and tox setup → identical for both runners

PREFER / AVOID

⇄ different flags locally and in CI → share them in `addopts` and add only CI-specific ones in the workflow

⇄ installing with `pip install -r` and no lock → `uv sync` or a lock file

⇄ one Python version → a matrix of supported versions

USEFUL CI FLAGS

FLAG	WHY
`--junitxml=path`	Test report for the CI UI
`-ra`	Reasons for skips / xfails
`-n auto`	Use all cores
`--cov-report=xml`	Coverage upload
`-W error`	Warnings fail
`--strict-markers`	Typos fail
`-p no:cacheprovider`	No cache dir

RUNNERS AROUND PYTEST

TOOL	USE
`tox`	Test matrix from tox.ini / tox.toml
`nox`	Matrix in Python (noxfile.py)
`uv run`	Run in the project env
`pre-commit`	Lint before tests
`Codecov / Coveralls`	Coverage dashboards

⚠ COMMON MISTAKES

- Ignoring exit code 5 when a path filter leaves nothing to run
- Uploading reports only on success, so failures have no report
- Testing only the newest Python while supporting older ones

✓ TIPS & BEST PRACTICES

- Use the same command locally and in CI (a `nox` session or a script)
- Fail on warnings in CI to catch deprecations before upgrades break you
- Shard huge suites with `pytest-split` or xdist across jobs

QUICK SUMMARY

- exit code — CI pass / fail
- `--junitxml` — reports
- matrix / tox / nox — many Pythons
- `-W error` + strict — fail early

Sheet 24

Quick Reference — Fixtures, Marks & Assertions

built-in fixtures · fixture options · marks · raises / warns / approx · parametrize forms

BUILT-IN FIXTURES (09, 13)

FIXTURE	GIVES
tmp_path / tmp_path_factory	Temp dirs
capsys / capfd	Captured output
caplog	Log records
monkeypatch	Undoable patches
recwarn	Recorded warnings
request	Test context, params
pytestconfig	Config, CLI options
subtests	Per-block reports (9.0)

FIXTURE OPTIONS (06–08)

OPTION	EFFECT
scope=	function · class · module · package · session
autouse=True	Apply automatically
params=[…]	Run per value
ids=	Names for params
name=	Register as another name
yield	Teardown after
request.param	Current param

ASSERTIONS (04, 05)

WRITE	CHECKS
assert a == b	Equality + diff
pytest.approx(v, rel=, abs=)	Floats
pytest.raises(E, match=)	Exception
pytest.RaisesGroup(E1, E2)	ExceptionGroup
pytest.warns(W)	Warning
pytest.deprecated_call()	Deprecation
excinfo.value	The exception

MARKS (10, 12)

MARK	EFFECT
skip(reason=)	Never run
skipif(cond, reason=)	Conditional skip
xfail(strict=True)	Known failure
parametrize("a,b", […])	Cases
usefixtures("x")	Apply fixtures
filterwarnings("…")	Warning filter
custom (slow…)	Select with -m

MOCKING (13, 14)

CALL	DOES
monkeypatch.setattr(t, v)	Swap attribute
monkeypatch.setenv(k, v)	Env var
mocker.patch(t, autospec=True)	Mock + undo
m.return_value / side_effect	Behavior
m.assert_called_once_with()	Check call
mocker.spy(obj, "fn")	Record real calls

PARAMETRIZE FORMS (10)

FORM	EXAMPLE
One arg	parametrize("n", [1, 2])
Many args	parametrize("a, b", [(1, 2)])
Named case	pytest.param(…, id="x")
Marked case	pytest.param(…, marks=…)
Indirect	indirect=True
Stacked	Cartesian product

SCOPES VS UNITTEST (07)

SCOPE	UNITTEST
function	setUp / tearDown
class	setUpClass
module	setUpModule
session	—
yield teardown	addCleanup

Sheet 25

Quick Reference — CLI & Config

selection flags · output flags · debug flags · config keys · common plugins and their flags

SELECTION (15)

FLAG	DOES
`path::Class::test[id]`	Node ID
`-k EXPR`	By name
`-m EXPR`	By mark
`--lf / --ff / --nf`	Last failed / failed first / new first
`-x / --maxfail=N`	Stop early
`--sw`	Stepwise
`--co`	Collect only
`--deselect ID`	Skip one

OUTPUT & DEBUG (16)

FLAG	DOES
`-v / -vv / -q`	Verbosity
`-s`	No capture
`-r fEsxXpA`	Summary
`--tb=short|line|no`	Traceback style
`-l`	Show locals
`--pdb / --trace`	Debugger
`--durations=N`	Slowest
`--setup-show`	Fixture setup / teardown

CONFIG KEYS (02, 03)

KEY	SETS
`testpaths`	Where to look
`addopts`	Default flags
`markers`	Registered marks
`strict`	All strict checks (9.0)
`filterwarnings`	Warning filters
`pythonpath`	Extra sys.path
`norecursedirs`	Skip folders
`log_cli / log_cli_level`	Live logs

PLUGINS (17–21)

PLUGIN	KEY FLAGS / MARKS
`pytest-cov`	--cov= --cov-report= --cov-branch --cov-fail-under=
`pytest-xdist`	-n auto --dist loadscope
`pytest-randomly`	-p no:randomly --randomly-seed=
`pytest-rerunfailures`	--reruns N, @flaky
`pytest-asyncio`	asyncio_mode = "auto"
`pytest-django`	@django_db --reuse-db
`pytest-mock`	mocker fixture
`syrupy`	--snapshot-update

EXIT CODES (01, 23)

CODE	MEANING
`0`	All passed
`1`	Failures
`2`	Interrupted
`3`	Internal error
`4`	Usage error
`5`	No tests collected

FILES (02, 08)

FILE	HOLDS
`pyproject.toml [tool.pytest]`	Config (9.0)
`pytest.toml`	Config (9.0)
`conftest.py`	Fixtures and hooks
`.pytest_cache/`	Last-run data (gitignore)
`__snapshots__/`	syrupy snapshots
`htmlcov/`	Coverage report

ENVIRONMENT VARIABLES

VARIABLE	EFFECT
`PYTEST_ADDOPTS`	Extra flags
`PYTEST_CURRENT_TEST`	Set during a test
`PYTHONBREAKPOINT=0`	Disable breakpoint()
`PYTEST_DISABLE_PLUGIN_AUTOLOAD`	No auto plugins
`CI`	Shorter output